Mini Project: Market Ups and Downs and Prediction of Stock Returns (Indian Market, 2000-2020)

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score,
confusion_matrix
```

In [2]:
```
df = pd.read_csv("Data/RELIANCE.NS.csv", parse_dates=["Date"])
df.head()
```

Out[2]:

	Date	Open	High	Low	Close	Adj Close	Volume
0	2000-01-03	36.747459	38.944569	36.747459	38.944569	29.684196	28802010.0
1	2000-01-04	39.981236	42.062302	38.882679	42.062302	32.060585	61320457.0
2	2000-01-05	39.710464	44.545658	39.710464	43.710136	33.316589	173426953.0
3	2000-01-06	44.715855	46.526150	44.715855	45.543640	34.714115	101355113.0
4	2000-01-07	45.644211	49.187443	45.334759	48.669109	37.096397	128426756.0

In [3]:
```
df.tail()
```

Out[3]:

	Date	Open	High	Low	Close	Adj Close	Volume
5125	2020-06-24	1735.900024	1772.699951	1711.699951	1727.849976	1710.716064	23807033.0
5126	2020-06-25	1727.000000	1749.000000	1713.050049	1717.900024	1700.864868	18312025.0
5127	2020-06-26	1728.800049	1752.000000	1717.000000	1741.650024	1724.379272	14270458.0
5128	2020-06-29	1728.050049	1739.750000	1715.550049	1723.150024	1706.062744	11824524.0
5129	2020-06-30	1737.000000	1741.000000	1695.550049	1704.099976	1687.201538	13972563.0

In [4]:
```
df.shape
```

Out[4]:
```
(5130, 7)
```

In [5]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5130 entries, 0 to 5129
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Date       5130 non-null   datetime64[us]
 1   Open       5120 non-null   float64
 2   High       5120 non-null   float64
 3   Low        5120 non-null   float64
 4   Close      5120 non-null   float64
 5   Adj Close  5120 non-null   float64
 6   Volume     5120 non-null   float64
dtypes: datetime64[us](1), float64(6)
memory usage: 280.7 KB
```

In [6]:
```
df.isnull().sum()
```

Out[6]:
```
Date          0
Open         10
High         10
Low          10
Close        10
Adj Close    10
Volume       10
dtype: int64
```

In [7]:
```
df = df.dropna().set_index("Date")
print("Rows with zero volume:", (df["Volume"] == 0).sum())
df[df["Volume"] == 0].head()
```

```
Rows with zero volume: 57
```

Out[7]:

Date	Open	High	Low	Close	Adj Close	Volume
2000-01-26	49.504631	49.504631	49.504631	49.504631	37.733246	0.0
2000-03-17	36.337437	36.337437	36.337437	36.337437	27.696993	0.0
2000-03-20	36.337437	36.337437	36.337437	36.337437	27.696993	0.0
2000-04-14	48.112095	48.112095	48.112095	48.112095	37.465439	0.0
2000-04-21	52.258755	52.258755	52.258755	52.258755	40.694489	0.0

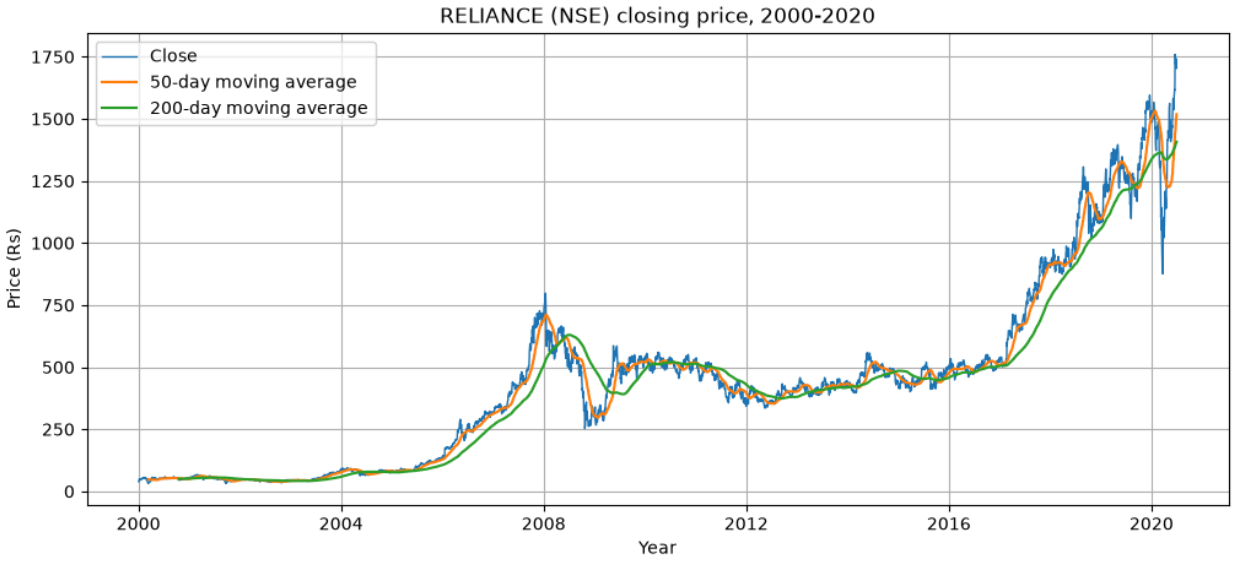
In [8]:
```
df = df[df["Volume"] > 0]
print("Rows after cleaning:", len(df))
df.describe()
```

```
Rows after cleaning: 5063
```

Out[8]:

	Open	High	Low	Close	Adj Close	Volume
count	5063.000000	5063.000000	5063.000000	5063.000000	5063.000000	5.063000e+03
mean	439.056195	444.772105	432.919165	438.622934	414.497285	1.790667e+07
std	361.403666	365.723595	356.813460	361.063140	358.173594	1.880613e+07
min	31.796223	33.962387	30.504259	31.440351	23.964357	2.132820e+05
25%	85.718285	87.033455	84.325752	85.288917	73.066788	6.660594e+06
50%	425.219330	430.667694	419.572876	424.278259	395.843262	1.098275e+07
75%	526.843567	531.957581	520.194092	525.543396	488.511383	2.243866e+07
max	1773.400024	1804.199951	1735.000000	1759.400024	1741.953369	2.918015e+08

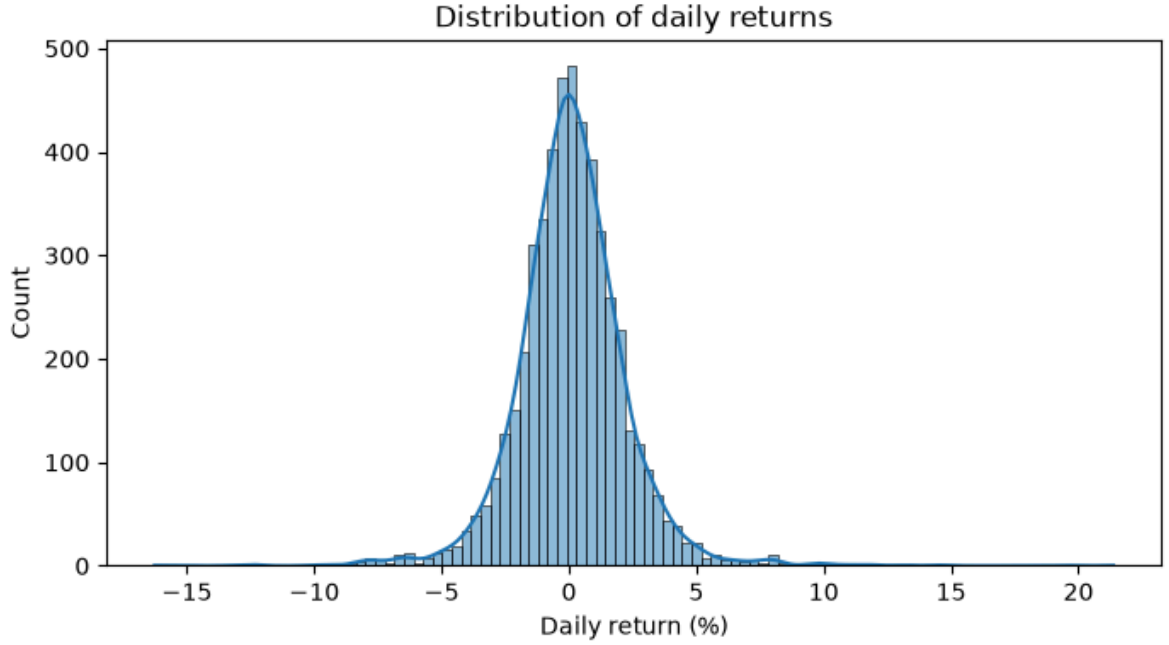
In [9]:
```
df["MA50"] = df["Close"].rolling(50).mean()
df["MA200"] = df["Close"].rolling(200).mean()
plt.figure(figsize=(12, 5))
plt.plot(df["Close"], label="Close", linewidth=1)
plt.plot(df["MA50"], label="50-day moving average")
plt.plot(df["MA200"], label="200-day moving average")
plt.title("RELIANCE (NSE) closing price, 2000-2020")
plt.xlabel("Year")
plt.ylabel("Price (Rs)")
plt.legend()
plt.grid(True)
plt.show()
```



In [10]:
```
df["Return"] = df["Adj Close"].pct_change()
df["Return"].describe()
```

Out[10]:
```
count    5062.000000
mean        0.001040
std         0.022011
min        -0.162732
25%        -0.010618
50%         0.000554
75%         0.011957
max         0.213693
Name: Return, dtype: float64
```

In [11]:
```
plt.figure(figsize=(8, 4))
sns.histplot(df["Return"].dropna() * 100, bins=100, kde=True)
plt.title("Distribution of daily returns")
plt.xlabel("Daily return (%)")
plt.show()
```



In [12]:
```
up_days = (df["Return"] > 0).sum()
down_days = (df["Return"] < 0).sum()
print("Up days  :", up_days)
print("Down days:", down_days)
print("Up days (%):", round(100 * up_days / (up_days + down_days), 2))
```

```
Up days  : 2612
Down days: 2443
Up days (%): 51.67
```

In [13]:
```python
print("Biggest daily rises (%):")
print((df["Return"].nlargest(5) * 100).round(2))
print()
print("Biggest daily falls (%):")
print((df["Return"].nsmallest(5) * 100).round(2))
```

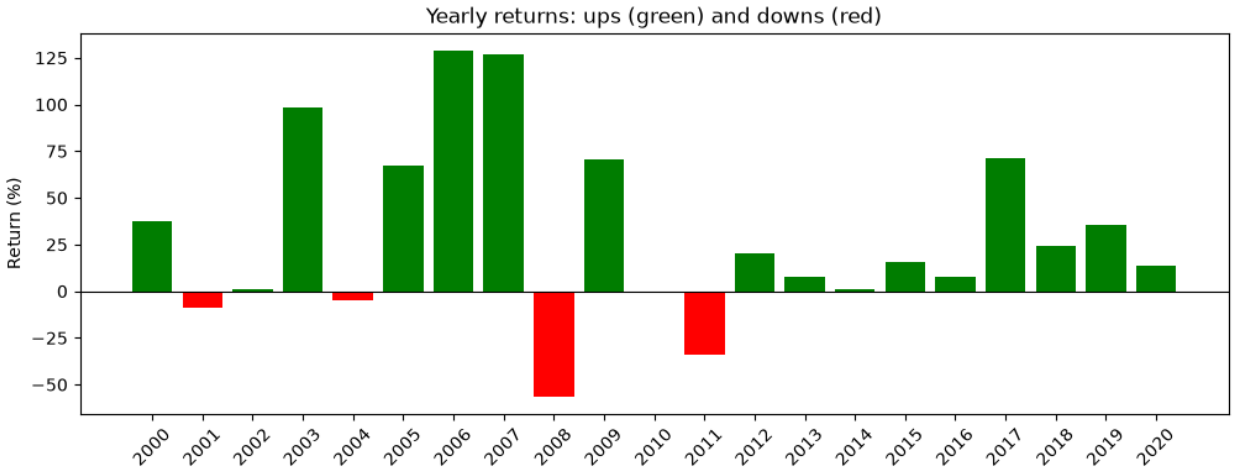
```
Biggest daily rises (%):
Date
2009-05-18    21.37
2006-01-18    19.58
2020-03-25    14.72
2008-10-31    14.45
2002-10-31    13.27
Name: Return, dtype: float64

Biggest daily falls (%):
Date
2008-10-24   -16.27
2004-05-17   -15.26
2020-03-23   -13.15
2008-11-05   -12.58
2009-01-07   -12.41
Name: Return, dtype: float64
```

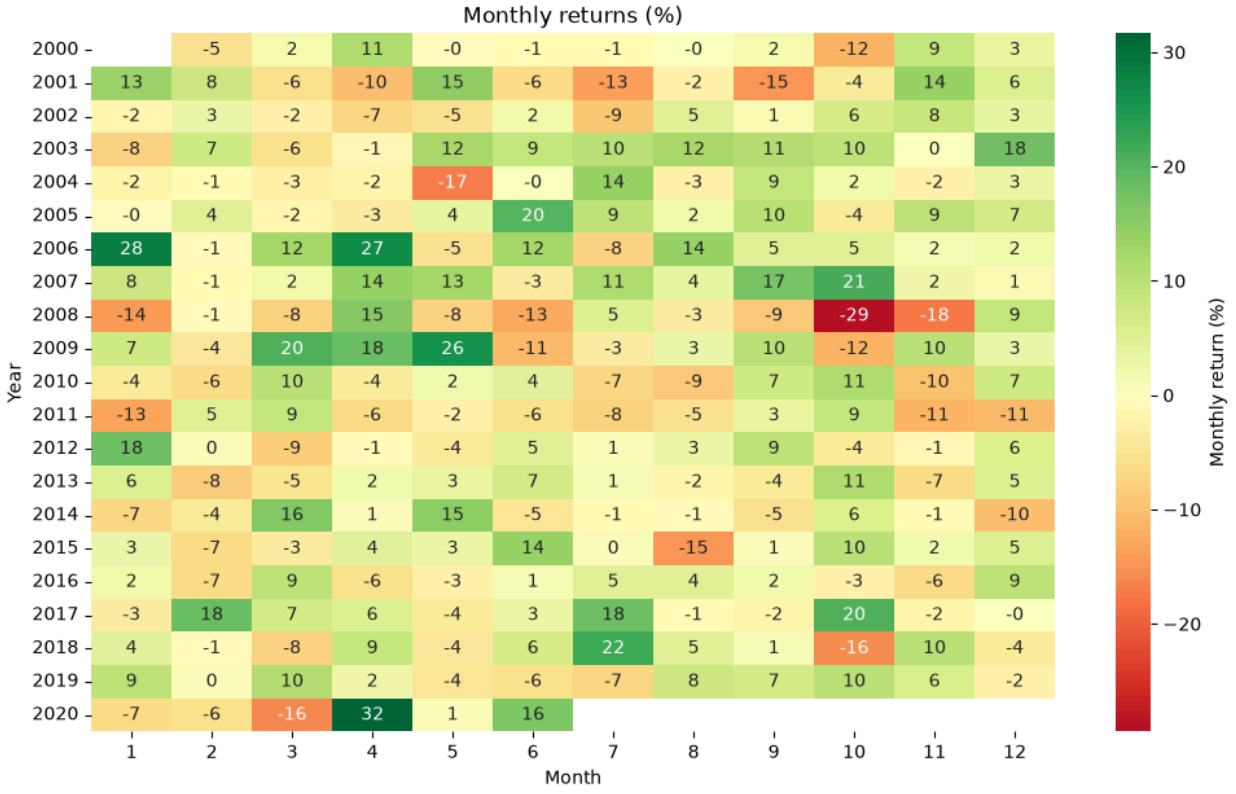
In [14]:
```python
yearly = df["Adj Close"].groupby(df.index.year).agg(["first", "last"])
yearly["Return (%)"] = ((yearly["last"] / yearly["first"] - 1) * 100).round(2)
print(yearly["Return (%)"].to_string())

colors = ["green" if r > 0 else "red" for r in yearly["Return (%)"]]
plt.figure(figsize=(12, 4))
plt.bar(yearly.index, yearly["Return (%)"], color=colors)
plt.axhline(0, color="black", linewidth=0.8)
plt.xticks(yearly.index, rotation=45)
plt.title("Yearly returns: ups (green) and downs (red)")
plt.ylabel("Return (%)")
plt.show()
```

```
Date
2000     37.58
2001     -8.67
2002      0.89
2003     98.14
2004     -4.71
2005     67.22
2006    128.83
2007    126.94
2008    -56.51
2009     70.57
2010     -0.89
2011    -33.80
2012     20.27
2013      7.67
2014      1.18
2015     15.60
2016      7.71
2017     71.01
2018     24.01
2019     35.81
2020     13.95
```

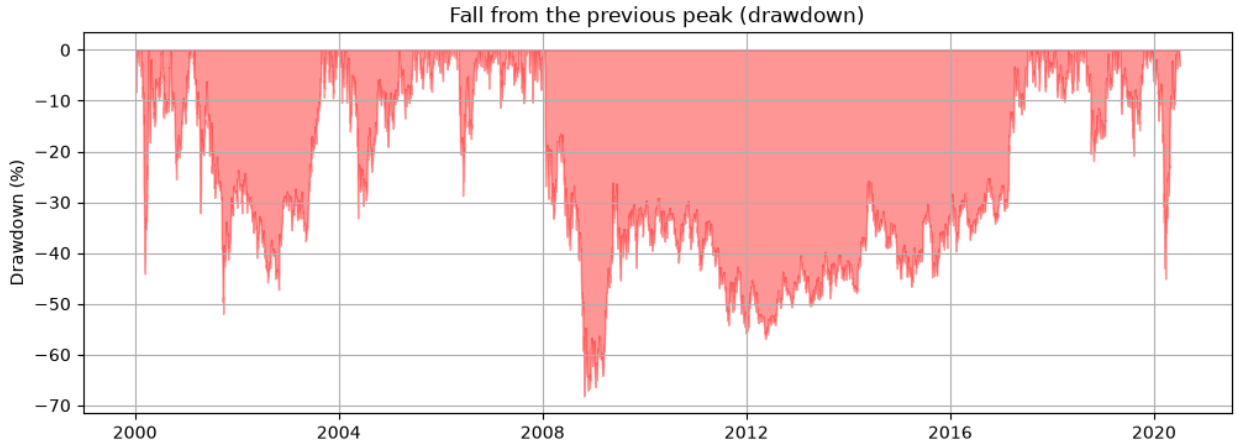


In [15]:
```
monthly = df["Adj Close"].resample("ME").last().pct_change() * 100
heat = pd.DataFrame({"Year": monthly.index.year, "Month": monthly.index.month, "Return":
monthly.values})
heat = heat.pivot(index="Year", columns="Month", values="Return")
plt.figure(figsize=(12, 7))
sns.heatmap(heat, cmap="RdYlGn", center=0, annot=True, fmt=".0f", cbar_kws={"label":
"Monthly return (%)"})
plt.title("Monthly returns (%)")
plt.show()
```

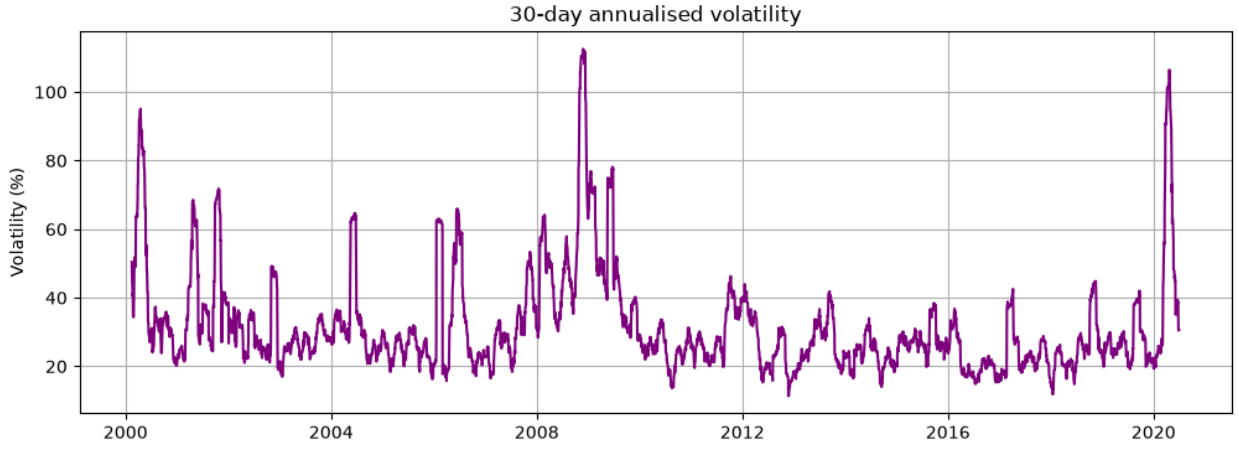


In [16]:
```
drawdown = (df["Adj Close"] / df["Adj Close"].cummax() - 1) * 100
print("Maximum drawdown:", round(drawdown.min(), 2), "% on", drawdown.idxmin().date())
plt.figure(figsize=(12, 4))
plt.fill_between(drawdown.index, drawdown, 0, color="red", alpha=0.4)
plt.title("Fall from the previous peak (drawdown)")
plt.ylabel("Drawdown (%)")
plt.grid(True)
plt.show()
```

```
Maximum drawdown: -68.19 % on 2008-10-24
```

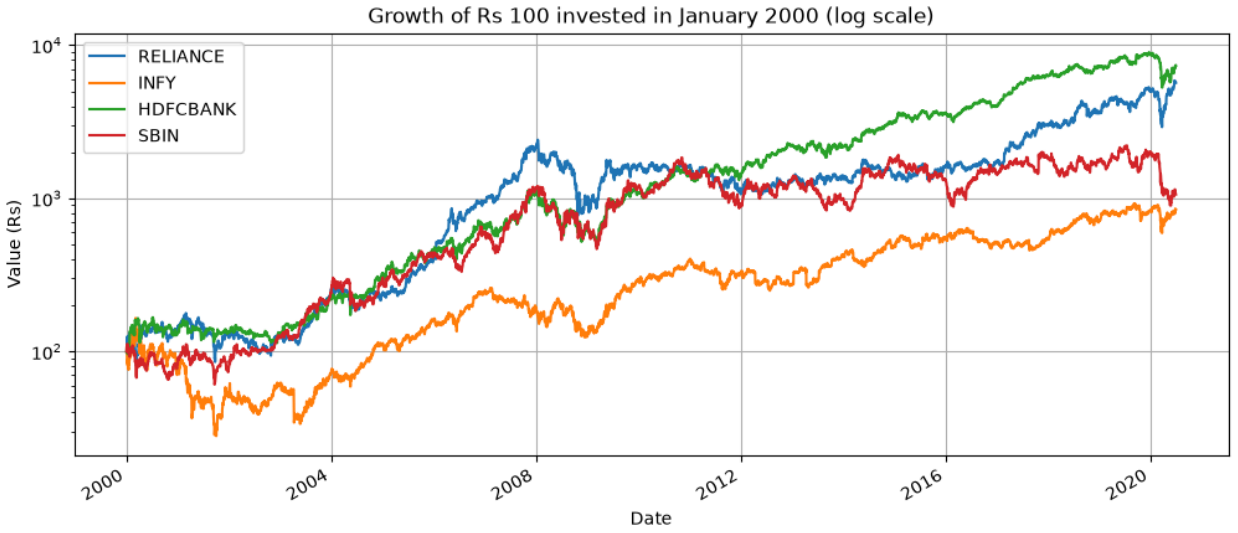


In [17]:
```
df["Volatility"] = df["Return"].rolling(30).std() * np.sqrt(252) * 100
plt.figure(figsize=(12, 4))
plt.plot(df["Volatility"], color="purple")
plt.title("30-day annualised volatility")
plt.ylabel("Volatility (%)")
plt.grid(True)
plt.show()
```



In [18]:
```
def load(symbol):
    data = pd.read_csv(f"Data/{symbol}.NS.csv", parse_dates=["Date"],
index_col="Date").dropna()
    return data[data["Volume"] > 0]["Adj Close"]

stocks = ["RELIANCE", "INFY", "HDFCBANK", "SBIN"]
prices = pd.DataFrame({s: load(s) for s in stocks}).dropna()
(prices / prices.iloc[0] * 100).plot(figsize=(12, 5), logy=True)
plt.title("Growth of Rs 100 invested in January 2000 (log scale)")
plt.ylabel("Value (Rs)")
plt.grid(True)
plt.show()
```

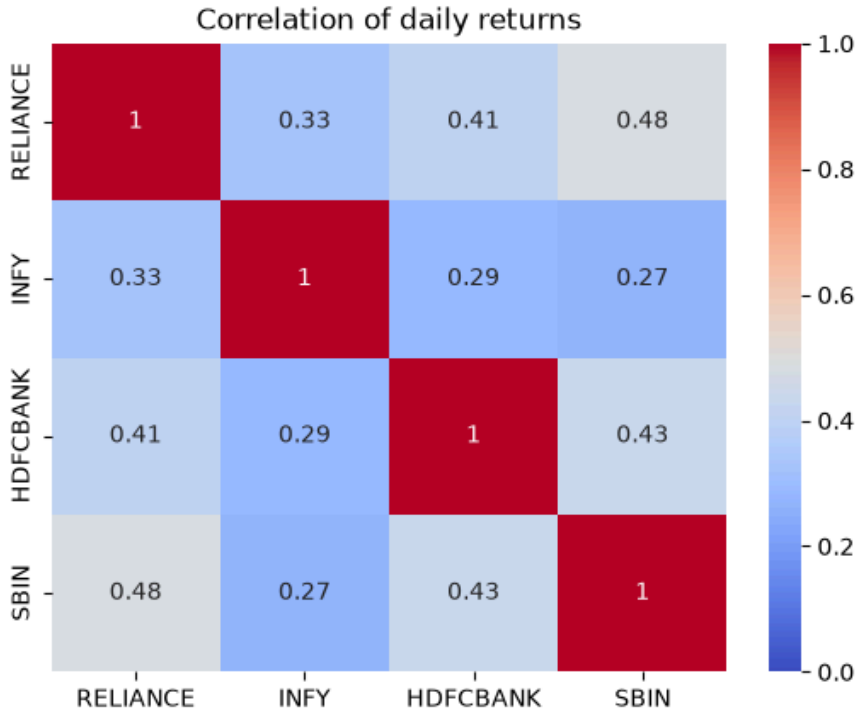


In [19]:
```
yearly_all = pd.DataFrame({s: (prices[s].groupby(prices.index.year).last() /
prices[s].groupby(prices.index.year).first() - 1) * 100 for s in stocks}).round(1)
yearly_all
```

Out[19]:

Date	RELIANCE	INFY	HDFCBANK	SBIN
2000	37.6	-27.1	32.1	-19.1
2001	-8.7	-26.7	2.4	-4.8
2002	0.9	18.7	-2.9	58.0
2003	98.1	17.0	71.4	93.0
2004	-4.7	51.7	43.2	18.6
2005	67.2	42.2	36.1	41.1
2006	128.8	52.9	50.6	40.2
2007	126.9	-21.6	62.7	91.2
2008	-56.5	-34.7	-42.0	-41.9
2009	70.6	132.7	68.7	73.2
2010	-0.9	33.9	38.4	24.4
2011	-33.8	-18.9	-10.1	-41.9
2012	20.3	-16.6	60.2	49.2
2013	7.7	53.6	-1.9	-25.8
2014	1.2	16.2	44.3	79.3
2015	15.6	14.7	14.5	-27.6
2016	7.7	-6.5	11.7	11.3
2017	71.0	7.2	57.5	28.4
2018	24.0	29.9	15.1	-3.6
2019	35.8	14.7	19.4	11.4
2020	14.0	1.2	-16.6	-46.6

In [20]:
```
sns.heatmap(prices.pct_change().dropna().corr(), annot=True, cmap="coolwarm", vmin=0,
vmax=1)
plt.title("Correlation of daily returns")
plt.show()
```



In [21]:
```
features = pd.DataFrame(index=df.index)
features["Return"] = df["Return"]
features["Return_lag1"] = df["Return"].shift(1)
features["Return_lag2"] = df["Return"].shift(2)
features["Return_5d"] = df["Adj Close"].pct_change(5)
features["Return_20d"] = df["Adj Close"].pct_change(20)
features["MA10_ratio"] = df["Adj Close"] / df["Adj Close"].rolling(10).mean()
features["MA50_ratio"] = df["Adj Close"] / df["Adj Close"].rolling(50).mean()
features["Volatility_20d"] = df["Return"].rolling(20).std()
features["Volume_change"] = np.log(df["Volume"] + 1).diff()
features = features.replace([np.inf, -np.inf], np.nan).dropna()

data = features.copy()
data["Target"] = df["Return"].shift(-1)
data = data.dropna()
data.head()
```

Out[21]:

Date	Return	Return_lag1	Return_lag2	Return_5d	Return_20d	MA10_ratio	MA50_ratio	Volatility_20d	Volume_change	Target
2000-03-13	-0.062947	-0.062473	-0.007083	-0.199843	-0.422891	0.791154	0.650790	0.033654	0.489697	0.078740
2000-03-14	0.078740	-0.062947	-0.062473	-0.095336	-0.363438	0.885031	0.703498	0.041090	-0.063177	0.080064
2000-03-15	0.080064	0.078740	-0.062947	0.016313	-0.319977	0.987817	0.761538	0.046510	-0.570728	-0.008026
2000-03-16	-0.008026	0.080064	0.078740	0.015348	-0.337611	1.003718	0.757749	0.045790	0.615047	0.012987
2000-03-21	0.012987	-0.008026	0.080064	0.097072	-0.319216	1.030204	0.770396	0.046353	-0.434979	0.023119

In [22]:
```
train = data[data.index < "2017-01-01"]
test = data[data.index >= "2017-01-01"]
cols = list(features.columns)
X_train, y_train = train[cols], train["Target"]
X_test, y_test = test[cols], test["Target"]
print("Training period:", train.index.min().date(), "to", train.index.max().date(),
X_train.shape)
print("Testing period :", test.index.min().date(), "to", test.index.max().date(),
X_test.shape)
```

```
Training period: 2000-03-13 to 2016-12-30 (4155, 9)
Testing period : 2017-01-02 to 2020-06-29 (858, 9)
```

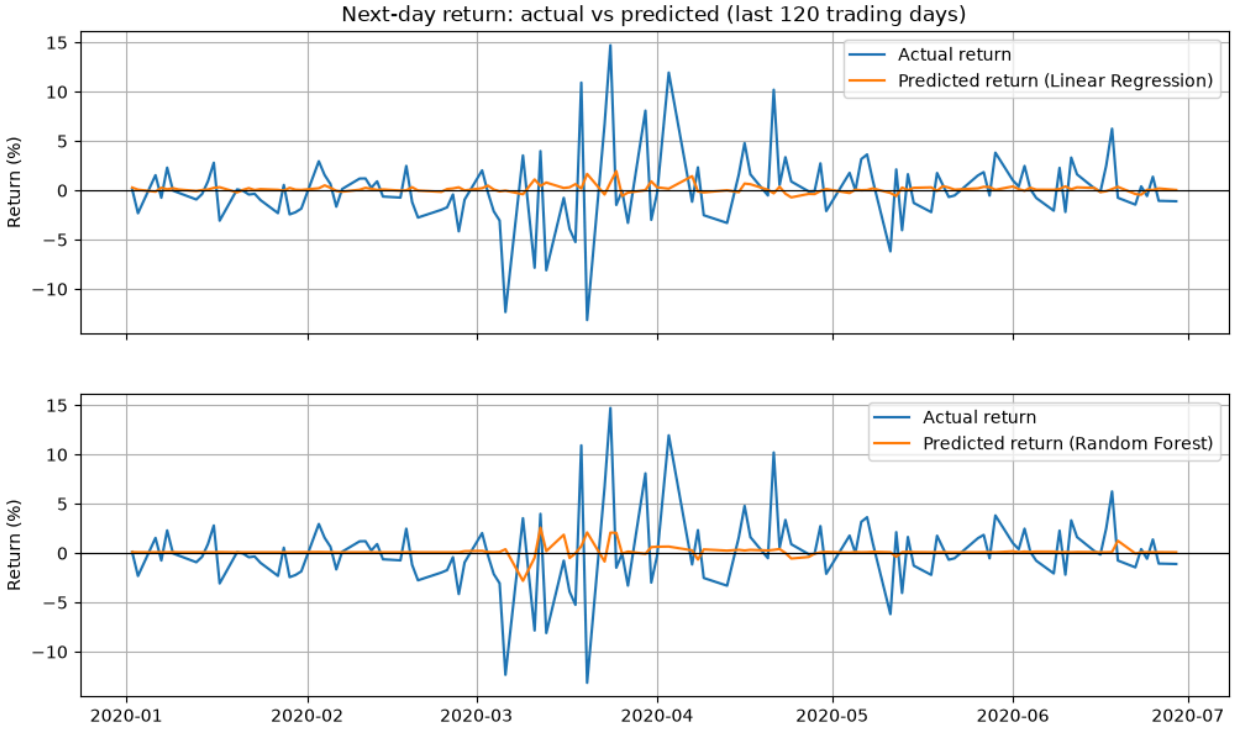
In [23]:
```
models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(n_estimators=300, max_depth=5, random_state=42)
}
results, predictions = [], {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    predictions[name] = pred
    results.append([name,
                    np.sqrt(mean_squared_error(y_test, pred)),
                    mean_absolute_error(y_test, pred),
                    r2_score(y_test, pred),
                    (np.sign(pred) == np.sign(y_test)).mean()])

zero = np.zeros(len(y_test))
results.append(["Baseline (no change, always 'up' for direction)",
                np.sqrt(mean_squared_error(y_test, zero)),
                mean_absolute_error(y_test, zero),
                r2_score(y_test, zero),
                (y_test > 0).mean()])
results = pd.DataFrame(results, columns=["Model", "RMSE", "MAE", "R2", "Direction
accuracy"]).round(4)
results
```

Out[23]:

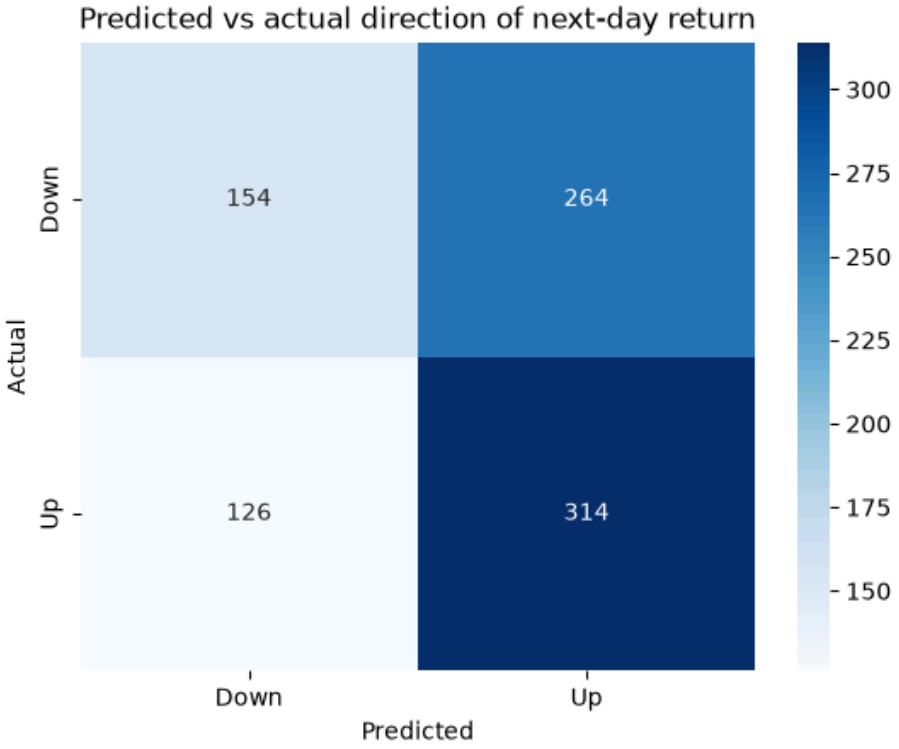
	Model	RMSE	MAE	R2	Direction accuracy
0	Linear Regression	0.0209	0.0140	-0.0267	0.5455
1	Random Forest	0.0207	0.0140	-0.0103	0.5128
2	Baseline (no change, always 'up' for direction)	0.0207	0.0139	-0.0059	0.5128

In [24]:
```
fig, ax = plt.subplots(2, 1, figsize=(12, 7), sharex=True)
for a, name in zip(ax, models):
    a.plot(y_test.index[-120:], y_test[-120:] * 100, label="Actual return")
    a.plot(y_test.index[-120:], predictions[name][-120:] * 100, label=f"Predicted return
({name})")
    a.axhline(0, color="black", linewidth=0.8)
    a.set_ylabel("Return (%)")
    a.legend()
    a.grid(True)
ax[0].set_title("Next-day return: actual vs predicted (last 120 trading days)")
plt.show()
```

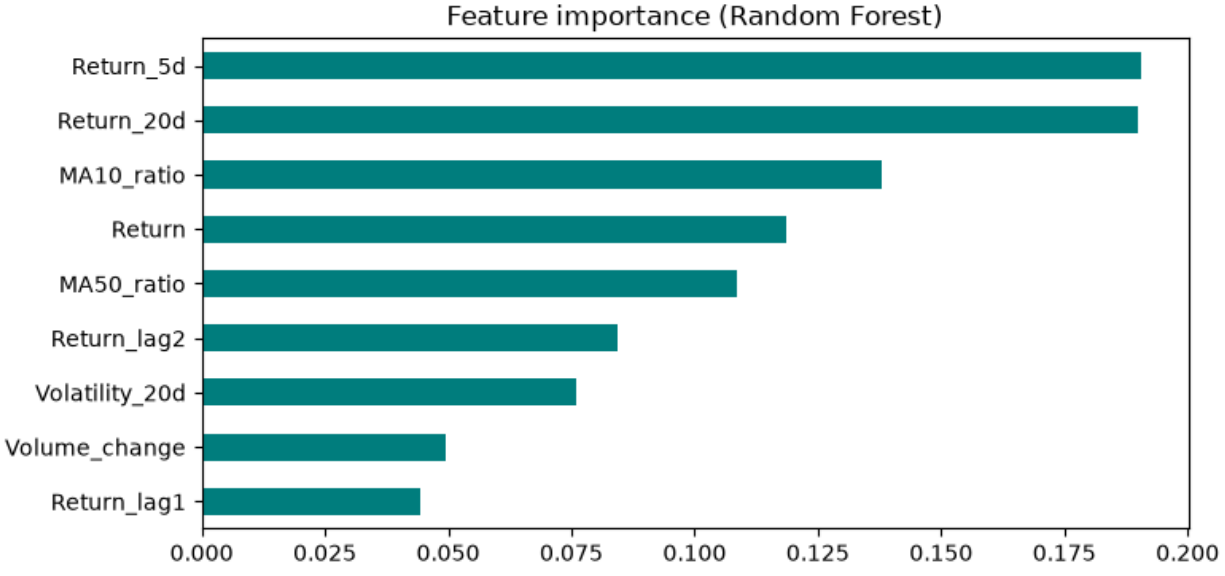


In [25]:
```
predicted_up = (predictions["Linear Regression"] > 0).astype(int)
actual_up = (y_test > 0).astype(int)
cm = confusion_matrix(actual_up, predicted_up)
print("Direction confusion matrix, Linear Regression (0 = down, 1 = up):")
print(cm)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=["Down", "Up"],
yticklabels=["Down", "Up"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Predicted vs actual direction of next-day return")
plt.show()
```

```
Direction confusion matrix, Linear Regression (0 = down, 1 = up):
[[154 264]
 [126 314]]
```



In [26]:
```
importance = pd.Series(models["Random Forest"].feature_importances_,
index=cols).sort_values()
importance.plot(kind="barh", figsize=(8, 4), color="teal")
plt.title("Feature importance (Random Forest)")
plt.show()
```



In [27]:
```
latest = features.iloc[[-1]]
last_price = df["Close"].iloc[-1]
print("Last date in the data  :", latest.index[0].date())
print("Last closing price (Rs):", round(last_price, 2))
for name, model in models.items():
    r = model.predict(latest)[0]
    print(f"{name:17s}: next-day return {r * 100:+.3f} %, expected price Rs {last_price *
(1 + r):.2f}, direction {'UP' if r > 0 else 'DOWN'}")
```

```
Last date in the data  : 2020-06-30
Last closing price (Rs): 1704.1
Linear Regression: next-day return -0.017 %, expected price Rs 1703.81, direction DOWN
Random Forest    : next-day return +0.083 %, expected price Rs 1705.52, direction UP
```

In [28]:
```
best = results.iloc[:2].sort_values("Direction accuracy", ascending=False).iloc[0]
print("Summary of the analysis")
print("Period analysed             :", df.index.min().date(), "to", df.index.max().date())
print("Total return of RELIANCE    :", round((df["Adj Close"].iloc[-1] / df["Adj
Close"].iloc[0] - 1) * 100, 1), "%")
print("Best year                   :", yearly["Return (%)"].idxmax(), f"({yearly['Return
(%)'].max()} %)")
print("Worst year                  :", yearly["Return (%)"].idxmin(), f"({yearly['Return
(%)'].min()} %)")
print("Down years for all 4 stocks :", list(yearly_all.index[(yearly_all <
0).all(axis=1)]))
print("Maximum drawdown            :", round(drawdown.min(), 2), "% on",
drawdown.idxmin().date())
print("Best model for direction    :", best["Model"], f"({best['Direction accuracy'] *
100:.2f} % correct)")
print("Baseline direction accuracy :", f"{(y_test > 0).mean() * 100:.2f} %")
```

```
Summary of the analysis
Period analysed             : 2000-01-03 to 2020-06-30
Total return of RELIANCE    : 5583.8 %
Best year                   : 2006 (128.83 %)
Worst year                  : 2008 (-56.51 %)
Down years for all 4 stocks : [2008, 2011]
Maximum drawdown            : -68.19 % on 2008-10-24
Best model for direction    : Linear Regression (54.55 % correct)
Baseline direction accuracy : 51.28 %
```